Contents

DSA Solutions Guide - Part 3: Trees & Graphs **2**
Complete JavaScript Solutions with Detailed Explanations . . . 2

TREES **2**
Tree Node Structure . . . 2

1. Tree Traversals **3**
Inorder (Left, Root, Right) . . . 3
Preorder (Root, Left, Right) . . . 3
Postorder (Left, Right, Root) . . . 4
Level Order (BFS) . . . 4

2. Maximum Depth of Binary Tree **5**
Problem . . . 5

3. Invert Binary Tree **5**
Problem . . . 5

4. Same Tree **6**
Problem . . . 6

5. Symmetric Tree **6**
Problem . . . 6

6. Diameter of Binary Tree **7**
Problem . . . 7

7. Balanced Binary Tree **8**
Problem . . . 8

8. Validate Binary Search Tree **8**
Problem . . . 8
Key Insight . . . 8

9. Kth Smallest Element in BST **9**
Problem . . . 9

10. Lowest Common Ancestor of BST **10**
Problem . . . 10

11. Lowest Common Ancestor of Binary Tree **11**
Problem . . . 11
Explanation . . . 11

12. Binary Tree Maximum Path Sum **11**
Problem . . . 11
Key Insight . . . 12

13. Construct Binary Tree from Preorder and Inorder **12**
Problem . . . 12
Explanation . . . 12

14. Serialize and Deserialize Binary Tree **13**
Problem . . . 13

15. Binary Tree Right Side View **13**
Problem . . . 13

GRAPHS **14**
Graph Representations . . . 14

16. Number of Islands **15**
Problem . . . 15

17. Clone Graph **16**
Problem . . . 16

18. Course Schedule (Cycle Detection) **17**
Problem . . . 17

19. Course Schedule II (Topological Sort) **18**
Problem . . . 18

20. Pacific Atlantic Water Flow **20**
Problem . . . 20

21. Word Ladder **21**
Problem . . . 21

22. Rotting Oranges **22**
Problem . . . 22

23. Network Delay Time (Dijkstra) **23**
Problem . . . 23

24. Union Find (Disjoint Set Union) **25**
Implementation . . . 25

25. Number of Connected Components **25**

26. Redundant Connection **26**
Problem . . . 26

Summary: Tree & Graph Patterns **27**
Tree Patterns . . . 27
Graph Patterns . . . 27

DSA Solutions Guide - Part 3: Trees & Graphs

Complete JavaScript Solutions with Detailed Explanations

TREES

Tree Node Structure

```javascript
class TreeNode {
    constructor(val = 0, left = null, right = null) {
        this.val = val;
        this.left = left;
        this.right = right;
    }
}
```

---

1. Tree Traversals

Inorder (Left, Root, Right)

```
// Recursive
function inorderRecursive(root) {
    const result = [];

    function traverse(node) {
        if (node === null) return;
        traverse(node.left);
        result.push(node.val);
        traverse(node.right);
    }

    traverse(root);
    return result;
}

// Iterative with Stack
function inorderIterative(root) {
    const result = [];
    const stack = [];
    let current = root;

    while (current !== null || stack.length > 0) {
        // Go to leftmost node
        while (current !== null) {
            stack.push(current);
            current = current.left;
        }

        // Process node
        current = stack.pop();
        result.push(current.val);

        // Move to right subtree
        current = current.right;
    }

    return result;
}

// Time: O(n), Space: O(h) where h = height
```

Preorder (Root, Left, Right)

```
// Iterative
function preorderIterative(root) {
    if (root === null) return [];

    const result = [];
    const stack = [root];

    while (stack.length > 0) {
        const node = stack.pop();
        result.push(node.val);
```

```
        // Push right first so left is processed first
        if (node.right) stack.push(node.right);
        if (node.left) stack.push(node.left);
    }

    return result;
}
```

Postorder (Left, Right, Root)

```
// Iterative with Two Stacks
function postorderIterative(root) {
    if (root === null) return [];

    const result = [];
    const stack1 = [root];
    const stack2 = [];

    while (stack1.length > 0) {
        const node = stack1.pop();
        stack2.push(node);

        if (node.left) stack1.push(node.left);
        if (node.right) stack1.push(node.right);
    }

    while (stack2.length > 0) {
        result.push(stack2.pop().val);
    }

    return result;
}
```

Level Order (BFS)

```
function levelOrder(root) {
    if (root === null) return [];

    const result = [];
    const queue = [root];

    while (queue.length > 0) {
        const levelSize = queue.length;
        const currentLevel = [];

        for (let i = 0; i < levelSize; i++) {
            const node = queue.shift();
            currentLevel.push(node.val);

            if (node.left) queue.push(node.left);
            if (node.right) queue.push(node.right);
        }

        result.push(currentLevel);
    }
```

```
    return result;
}

// Time: O(n), Space: O(w) where w = max width
```

---

2. Maximum Depth of Binary Tree

Problem

Find the maximum depth (number of nodes along longest path from root to leaf).

```
// Recursive DFS
function maxDepth(root) {
    if (root === null) return 0;

    const leftDepth = maxDepth(root.left);
    const rightDepth = maxDepth(root.right);

    return 1 + Math.max(leftDepth, rightDepth);
}

// Iterative BFS
function maxDepthBFS(root) {
    if (root === null) return 0;

    const queue = [root];
    let depth = 0;

    while (queue.length > 0) {
        const levelSize = queue.length;
        depth++;

        for (let i = 0; i < levelSize; i++) {
            const node = queue.shift();
            if (node.left) queue.push(node.left);
            if (node.right) queue.push(node.right);
        }
    }

    return depth;
}

// Time: O(n), Space: O(h)
```

---

3. Invert Binary Tree

Problem

Mirror the tree (swap left and right children for all nodes).

```
function invertTree(root) {
    if (root === null) return null;

    // Swap children
```

```
    const temp = root.left;
    root.left = root.right;
    root.right = temp;

    // Recursively invert subtrees
    invertTree(root.left);
    invertTree(root.right);

    return root;
}

// One-liner
function invertTreeOneLine(root) {
    if (root === null) return null;
    [root.left, root.right] = [invertTree(root.right), invertTree(root.left)];
    return root;
}

// Time: O(n), Space: O(h)
```

---

4. Same Tree

Problem

Check if two binary trees are identical.

```
function isSameTree(p, q) {
    // Both null
    if (p === null && q === null) return true;

    // One null, one not
    if (p === null || q === null) return false;

    // Values different
    if (p.val !== q.val) return false;

    // Recursively check subtrees
    return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
}

// Time: O(min(m, n)), Space: O(min(h1, h2))
```

---

5. Symmetric Tree

Problem

Check if a tree is symmetric around its center.

```
function isSymmetric(root) {
    if (root === null) return true;
    return isMirror(root.left, root.right);
}

function isMirror(left, right) {
```

```
    if (left === null && right === null) return true;
    if (left === null || right === null) return false;

    return (
        left.val === right.val &&
        isMirror(left.left, right.right) &&
        isMirror(left.right, right.left)
    );
}

// Iterative with Queue
function isSymmetricIterative(root) {
    if (root === null) return true;

    const queue = [root.left, root.right];

    while (queue.length > 0) {
        const left = queue.shift();
        const right = queue.shift();

        if (left === null && right === null) continue;
        if (left === null || right === null) return false;
        if (left.val !== right.val) return false;

        queue.push(left.left, right.right);
        queue.push(left.right, right.left);
    }

    return true;
}

// Time: O(n), Space: O(h)
```

---

6. Diameter of Binary Tree

Problem

Find the length of the longest path between any two nodes.

```
function diameterOfBinaryTree(root) {
    let maxDiameter = 0;

    function height(node) {
        if (node === null) return 0;

        const leftHeight = height(node.left);
        const rightHeight = height(node.right);

        // Update diameter (path through this node)
        maxDiameter = Math.max(maxDiameter, leftHeight + rightHeight);

        return 1 + Math.max(leftHeight, rightHeight);
    }

    height(root);
```

```
    return maxDiameter;
}

// Time: O(n), Space: O(h)
```

---

7. Balanced Binary Tree

Problem

Check if tree is height-balanced (heights of subtrees differ by at most 1).

```
function isBalanced(root) {
    return checkHeight(root) !== -1;
}

function checkHeight(node) {
    if (node === null) return 0;

    const leftHeight = checkHeight(node.left);
    if (leftHeight === -1) return -1;

    const rightHeight = checkHeight(node.right);
    if (rightHeight === -1) return -1;

    if (Math.abs(leftHeight - rightHeight) > 1) return -1;

    return 1 + Math.max(leftHeight, rightHeight);
}

// Time: O(n), Space: O(h)
```

---

8. Validate Binary Search Tree

Problem

Determine if a tree is a valid BST.

Key Insight

Each node must be within a valid range: (min, max)

```
function isValidBST(root) {
    return validate(root, -Infinity, Infinity);
}

function validate(node, min, max) {
    if (node === null) return true;

    // Check current node's value
    if (node.val <= min || node.val >= max) {
        return false;
    }

    // Left subtree: all values must be less than current
```

```
    // Right subtree: all values must be greater than current
    return (
        validate(node.left, min, node.val) &&
        validate(node.right, node.val, max)
    );
}

// Alternative: Inorder traversal should be sorted
function isValidBSTInorder(root) {
    let prev = -Infinity;

    function inorder(node) {
        if (node === null) return true;

        if (!inorder(node.left)) return false;

        if (node.val <= prev) return false;
        prev = node.val;

        return inorder(node.right);
    }

    return inorder(root);
}

// Time: O(n), Space: O(h)
```

---

9. Kth Smallest Element in BST

Problem

Find the kth smallest element in a BST.

```
function kthSmallest(root, k) {
    let count = 0;
    let result = null;

    function inorder(node) {
        if (node === null || result !== null) return;

        inorder(node.left);

        count++;
        if (count === k) {
            result = node.val;
            return;
        }

        inorder(node.right);
    }

    inorder(root);
    return result;
}
```

```
// Iterative
function kthSmallestIterative(root, k) {
    const stack = [];
    let current = root;

    while (current !== null || stack.length > 0) {
        while (current !== null) {
            stack.push(current);
            current = current.left;
        }

        current = stack.pop();
        k--;

        if (k === 0) return current.val;

        current = current.right;
    }

    return -1;
}

// Time: O(h + k), Space: O(h)
```

---

10. Lowest Common Ancestor of BST

Problem

Find the LCA of two nodes in a BST.

```
function lowestCommonAncestorBST(root, p, q) {
    // Use BST property
    while (root !== null) {
        if (p.val < root.val && q.val < root.val) {
            // Both in left subtree
            root = root.left;
        } else if (p.val > root.val && q.val > root.val) {
            // Both in right subtree
            root = root.right;
        } else {
            // Split point found (or one is ancestor of other)
            return root;
        }
    }

    return null;
}

// Time: O(h), Space: O(1)
```

---

11. Lowest Common Ancestor of Binary Tree

Problem

Find the LCA of two nodes in a binary tree.

```
function lowestCommonAncestor(root, p, q) {
    if (root === null || root === p || root === q) {
        return root;
    }

    const left = lowestCommonAncestor(root.left, p, q);
    const right = lowestCommonAncestor(root.right, p, q);

    // If both sides found something, current node is LCA
    if (left !== null && right !== null) {
        return root;
    }

    // Otherwise, return the non-null result
    return left !== null ? left : right;
}

// Time: O(n), Space: O(h)
```

Explanation

- If current node is p or q, return it
- Search both subtrees
- If both return non-null, current is LCA
- Otherwise, LCA is in the subtree that returned non-null

---

12. Binary Tree Maximum Path Sum

Problem

Find the maximum path sum (path can start and end at any node).

```
function maxPathSum(root) {
    let maxSum = -Infinity;

    function maxGain(node) {
        if (node === null) return 0;

        // Maximum gain from left and right (ignore negative gains)
        const leftGain = Math.max(0, maxGain(node.left));
        const rightGain = Math.max(0, maxGain(node.right));

        // Path through current node
        const pathSum = node.val + leftGain + rightGain;
        maxSum = Math.max(maxSum, pathSum);

        // Return max gain if continuing path through this node
        return node.val + Math.max(leftGain, rightGain);
    }
```

```
    maxGain(root);
    return maxSum;
}

// Time: O(n), Space: O(h)
```

Key Insight

- At each node, we calculate: path through this node (left + node + right)
- But we can only return one direction to parent (can't fork)

---

13. Construct Binary Tree from Preorder and Inorder

Problem

Build tree from preorder and inorder traversals.

```
function buildTree(preorder, inorder) {
    const inorderMap = new Map();
    inorder.forEach((val, idx) => inorderMap.set(val, idx));

    let preorderIdx = 0;

    function build(left, right) {
        if (left > right) return null;

        // Root is next element in preorder
        const rootVal = preorder[preorderIdx++];
        const root = new TreeNode(rootVal);

        // Find root position in inorder
        const inorderIdx = inorderMap.get(rootVal);

        // Build left subtree (elements before root in inorder)
        root.left = build(left, inorderIdx - 1);
        // Build right subtree (elements after root in inorder)
        root.right = build(inorderIdx + 1, right);

        return root;
    }

    return build(0, inorder.length - 1);
}

// Time: O(n), Space: O(n)
```

Explanation

- Preorder: first element is always root
- Inorder: elements left of root are in left subtree, right of root are in right subtree

---

14. Serialize and Deserialize Binary Tree

Problem

Design an algorithm to serialize and deserialize a binary tree.

```
class Codec {
    // Encodes a tree to a string
    serialize(root) {
        const result = [];

        function preorder(node) {
            if (node === null) {
                result.push('null');
                return;
            }
            result.push(node.val.toString());
            preorder(node.left);
            preorder(node.right);
        }

        preorder(root);
        return result.join(',');
    }

    // Decodes a string to a tree
    deserialize(data) {
        const values = data.split(',');
        let index = 0;

        function build() {
            if (values[index] === 'null') {
                index++;
                return null;
            }

            const node = new TreeNode(parseInt(values[index++]));
            node.left = build();
            node.right = build();
            return node;
        }

        return build();
    }
}

// Time: O(n), Space: O(n)
```

---

15. Binary Tree Right Side View

Problem

Return values visible from the right side of the tree.

```
function rightSideView(root) {
    if (root === null) return [];
```

```
    const result = [];
    const queue = [root];

    while (queue.length > 0) {
        const levelSize = queue.length;

        for (let i = 0; i < levelSize; i++) {
            const node = queue.shift();

            // Last node in level is visible from right
            if (i === levelSize - 1) {
                result.push(node.val);
            }

            if (node.left) queue.push(node.left);
            if (node.right) queue.push(node.right);
        }
    }

    return result;
}

// DFS approach - visit right first
function rightSideViewDFS(root) {
    const result = [];

    function dfs(node, depth) {
        if (node === null) return;

        // First node at this depth (from right)
        if (depth === result.length) {
            result.push(node.val);
        }

        dfs(node.right, depth + 1);  // Right first
        dfs(node.left, depth + 1);
    }

    dfs(root, 0);
    return result;
}

// Time: O(n), Space: O(h)
```

GRAPHS

Graph Representations

```
// Adjacency List (most common)
const graph = {
    0: [1, 2],
    1: [0, 3],
    2: [0, 3],
    3: [1, 2]
```

```
};

// Or using Map
const graphMap = new Map();
graphMap.set(0, [1, 2]);
graphMap.set(1, [0, 3]);

// Build from edge list
function buildGraph(edges, n, directed = false) {
    const graph = new Map();
    for (let i = 0; i < n; i++) {
        graph.set(i, []);
    }
    for (const [u, v] of edges) {
        graph.get(u).push(v);
        if (!directed) {
            graph.get(v).push(u);
        }
    }
    return graph;
}
```

---

16. Number of Islands

Problem

Count the number of islands in a 2D grid of '1's (land) and '0's (water).

```
function numIslands(grid) {
    if (grid.length === 0) return 0;

    const rows = grid.length;
    const cols = grid[0].length;
    let count = 0;

    function dfs(r, c) {
        // Boundary check and water check
        if (r < 0 || r >= rows || c < 0 || c >= cols || grid[r][c] === '0') {
            return;
        }

        // Mark as visited
        grid[r][c] = '0';

        // Explore all 4 directions
        dfs(r + 1, c);
        dfs(r - 1, c);
        dfs(r, c + 1);
        dfs(r, c - 1);
    }

    for (let r = 0; r < rows; r++) {
        for (let c = 0; c < cols; c++) {
            if (grid[r][c] === '1') {
                count++;
```

```
                dfs(r, c);  // Sink the island
            }
        }
    }

    return count;
}

// BFS Version
function numIslandsBFS(grid) {
    if (grid.length === 0) return 0;

    const rows = grid.length;
    const cols = grid[0].length;
    const directions = [[1, 0], [-1, 0], [0, 1], [0, -1]];
    let count = 0;

    for (let r = 0; r < rows; r++) {
        for (let c = 0; c < cols; c++) {
            if (grid[r][c] === '1') {
                count++;

                const queue = [[r, c]];
                grid[r][c] = '0';

                while (queue.length > 0) {
                    const [row, col] = queue.shift();

                    for (const [dr, dc] of directions) {
                        const nr = row + dr;
                        const nc = col + dc;

                        if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && grid[nr][nc] === '1') {
                            queue.push([nr, nc]);
                            grid[nr][nc] = '0';
                        }
                    }
                }
            }
        }
    }

    return count;
}

// Time: O(m * n), Space: O(m * n) worst case
```

---

17. Clone Graph

Problem

Deep copy a connected undirected graph.

```
function cloneGraph(node) {
    if (node === null) return null;
```

```
    const visited = new Map();  // Original node -> Cloned node

    function dfs(original) {
        if (visited.has(original)) {
            return visited.get(original);
        }

        // Create clone
        const clone = new Node(original.val);
        visited.set(original, clone);

        // Clone neighbors
        for (const neighbor of original.neighbors) {
            clone.neighbors.push(dfs(neighbor));
        }

        return clone;
    }

    return dfs(node);
}

// BFS Version
function cloneGraphBFS(node) {
    if (node === null) return null;

    const visited = new Map();
    const queue = [node];
    visited.set(node, new Node(node.val));

    while (queue.length > 0) {
        const current = queue.shift();

        for (const neighbor of current.neighbors) {
            if (!visited.has(neighbor)) {
                visited.set(neighbor, new Node(neighbor.val));
                queue.push(neighbor);
            }
            visited.get(current).neighbors.push(visited.get(neighbor));
        }
    }

    return visited.get(node);
}

// Time: O(V + E), Space: O(V)
```

---

18. Course Schedule (Cycle Detection)

Problem

Determine if you can finish all courses given prerequisites. This is cycle detection in a directed graph.

```javascript
function canFinish(numCourses, prerequisites) {
    // Build adjacency list
    const graph = new Map();
    for (let i = 0; i < numCourses; i++) {
        graph.set(i, []);
    }
    for (const [course, prereq] of prerequisites) {
        graph.get(prereq).push(course);
    }

    // 0 = unvisited, 1 = visiting, 2 = visited
    const state = new Array(numCourses).fill(0);

    function hasCycle(course) {
        if (state[course] === 1) return true;   // Cycle detected
        if (state[course] === 2) return false;  // Already processed

        state[course] = 1;  // Mark as visiting

        for (const next of graph.get(course)) {
            if (hasCycle(next)) return true;
        }

        state[course] = 2;  // Mark as visited
        return false;
    }

    // Check all courses (graph may be disconnected)
    for (let i = 0; i < numCourses; i++) {
        if (hasCycle(i)) return false;
    }

    return true;
}

// Time: O(V + E), Space: O(V + E)
```

---

19. Course Schedule II (Topological Sort)

Problem

Return the ordering of courses to finish all courses (topological sort).

```javascript
// Kahn's Algorithm (BFS)
function findOrder(numCourses, prerequisites) {
    const graph = new Map();
    const indegree = new Array(numCourses).fill(0);

    // Build graph and compute indegrees
    for (let i = 0; i < numCourses; i++) {
        graph.set(i, []);
    }
    for (const [course, prereq] of prerequisites) {
        graph.get(prereq).push(course);
        indegree[course]++;
```

```
    }

    // Start with courses having no prerequisites
    const queue = [];
    for (let i = 0; i < numCourses; i++) {
        if (indegree[i] === 0) {
            queue.push(i);
        }
    }

    const result = [];

    while (queue.length > 0) {
        const course = queue.shift();
        result.push(course);

        for (const next of graph.get(course)) {
            indegree[next]--;
            if (indegree[next] === 0) {
                queue.push(next);
            }
        }
    }

    // If we processed all courses, return result
    return result.length === numCourses ? result : [];
}

// DFS Approach
function findOrderDFS(numCourses, prerequisites) {
    const graph = new Map();
    for (let i = 0; i < numCourses; i++) {
        graph.set(i, []);
    }
    for (const [course, prereq] of prerequisites) {
        graph.get(prereq).push(course);
    }

    const state = new Array(numCourses).fill(0);
    const result = [];

    function dfs(course) {
        if (state[course] === 1) return false;  // Cycle
        if (state[course] === 2) return true;   // Already processed

        state[course] = 1;

        for (const next of graph.get(course)) {
            if (!dfs(next)) return false;
        }

        state[course] = 2;
        result.unshift(course);  // Add to front (reverse postorder)
        return true;
    }
```

```
    for (let i = 0; i < numCourses; i++) {
        if (!dfs(i)) return [];
    }

    return result;
}

// Time: O(V + E), Space: O(V + E)
```

---

20. Pacific Atlantic Water Flow

Problem

Find cells that can flow to both Pacific and Atlantic oceans.

```
function pacificAtlantic(heights) {
    if (heights.length === 0) return [];

    const rows = heights.length;
    const cols = heights[0].length;

    const pacific = Array.from({ length: rows }, () => new Array(cols).fill(false));
    const atlantic = Array.from({ length: rows }, () => new Array(cols).fill(false));

    const directions = [[1, 0], [-1, 0], [0, 1], [0, -1]];

    function dfs(r, c, reachable, prevHeight) {
        if (r < 0 || r >= rows || c < 0 || c >= cols) return;
        if (reachable[r][c]) return;  // Already visited
        if (heights[r][c] < prevHeight) return;  // Water can't flow uphill

        reachable[r][c] = true;

        for (const [dr, dc] of directions) {
            dfs(r + dr, c + dc, reachable, heights[r][c]);
        }
    }

    // Start DFS from ocean borders
    for (let c = 0; c < cols; c++) {
        dfs(0, c, pacific, 0);           // Top row (Pacific)
        dfs(rows - 1, c, atlantic, 0);   // Bottom row (Atlantic)
    }
    for (let r = 0; r < rows; r++) {
        dfs(r, 0, pacific, 0);           // Left column (Pacific)
        dfs(r, cols - 1, atlantic, 0);   // Right column (Atlantic)
    }

    // Find cells reachable from both
    const result = [];
    for (let r = 0; r < rows; r++) {
        for (let c = 0; c < cols; c++) {
            if (pacific[r][c] && atlantic[r][c]) {
                result.push([r, c]);
            }
```

```
        }
    }

    return result;
}

// Time: O(m * n), Space: O(m * n)
```

---

21. Word Ladder

Problem

Find the shortest transformation sequence from beginWord to endWord.

```
function ladderLength(beginWord, endWord, wordList) {
    const wordSet = new Set(wordList);
    if (!wordSet.has(endWord)) return 0;

    const queue = [[beginWord, 1]];
    const visited = new Set([beginWord]);

    while (queue.length > 0) {
        const [word, level] = queue.shift();

        if (word === endWord) return level;

        // Try changing each character
        for (let i = 0; i < word.length; i++) {
            for (let c = 97; c <= 122; c++) {  // a-z
                const newChar = String.fromCharCode(c);
                const newWord = word.slice(0, i) + newChar + word.slice(i + 1);

                if (wordSet.has(newWord) && !visited.has(newWord)) {
                    visited.add(newWord);
                    queue.push([newWord, level + 1]);
                }
            }
        }
    }

    return 0;
}

// Bidirectional BFS (Optimized)
function ladderLengthBidirectional(beginWord, endWord, wordList) {
    const wordSet = new Set(wordList);
    if (!wordSet.has(endWord)) return 0;

    let beginSet = new Set([beginWord]);
    let endSet = new Set([endWord]);
    let level = 1;

    while (beginSet.size > 0 && endSet.size > 0) {
        // Always expand the smaller set
        if (beginSet.size > endSet.size) {
```

```
            [beginSet, endSet] = [endSet, beginSet];
        }

        const nextSet = new Set();

        for (const word of beginSet) {
            for (let i = 0; i < word.length; i++) {
                for (let c = 97; c <= 122; c++) {
                    const newWord = word.slice(0, i) + String.fromCharCode(c) + word.slice(i + 1);

                    if (endSet.has(newWord)) {
                        return level + 1;
                    }

                    if (wordSet.has(newWord)) {
                        nextSet.add(newWord);
                        wordSet.delete(newWord);
                    }
                }
            }
        }

        beginSet = nextSet;
        level++;
    }

    return 0;
}

// Time: O(n * m * 26) where n = wordList length, m = word length
// Space: O(n * m)
```

---

22. Rotting Oranges

Problem

Return minimum minutes for all oranges to rot (BFS multi-source).

```
function orangesRotting(grid) {
    const rows = grid.length;
    const cols = grid[0].length;
    const queue = [];
    let freshCount = 0;

    // Find all rotten oranges and count fresh
    for (let r = 0; r < rows; r++) {
        for (let c = 0; c < cols; c++) {
            if (grid[r][c] === 2) {
                queue.push([r, c]);
            } else if (grid[r][c] === 1) {
                freshCount++;
            }
        }
    }
```

```
    if (freshCount === 0) return 0;

    const directions = [[1, 0], [-1, 0], [0, 1], [0, -1]];
    let minutes = 0;

    while (queue.length > 0 && freshCount > 0) {
        minutes++;
        const size = queue.length;

        for (let i = 0; i < size; i++) {
            const [r, c] = queue.shift();

            for (const [dr, dc] of directions) {
                const nr = r + dr;
                const nc = c + dc;

                if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && grid[nr][nc] === 1) {
                    grid[nr][nc] = 2;
                    freshCount--;
                    queue.push([nr, nc]);
                }
            }
        }
    }

    return freshCount === 0 ? minutes : -1;
}

// Time: O(m * n), Space: O(m * n)
```

---

23. Network Delay Time (Dijkstra)

Problem

Time for signal to reach all nodes from source (shortest path in weighted graph).

```
function networkDelayTime(times, n, k) {
    // Build adjacency list
    const graph = new Map();
    for (let i = 1; i <= n; i++) {
        graph.set(i, []);
    }
    for (const [u, v, w] of times) {
        graph.get(u).push([v, w]);
    }

    // Dijkstra's algorithm
    const dist = new Array(n + 1).fill(Infinity);
    dist[k] = 0;

    // Min-heap: [distance, node]
    const minHeap = new MinPriorityQueue({ priority: x => x[0] });
    minHeap.enqueue([0, k]);

    while (!minHeap.isEmpty()) {
```

```
        const [d, u] = minHeap.dequeue().element;

        if (d > dist[u]) continue;  // Already found better path

        for (const [v, w] of graph.get(u)) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                minHeap.enqueue([dist[v], v]);
            }
        }
    }

    // Find maximum distance
    const maxDist = Math.max(...dist.slice(1));
    return maxDist === Infinity ? -1 : maxDist;
}

// Simple implementation without library
function networkDelayTimeSimple(times, n, k) {
    const graph = new Map();
    for (let i = 1; i <= n; i++) graph.set(i, []);
    for (const [u, v, w] of times) graph.get(u).push([v, w]);

    const dist = new Array(n + 1).fill(Infinity);
    dist[k] = 0;

    const visited = new Set();

    while (visited.size < n) {
        // Find unvisited node with minimum distance
        let minDist = Infinity;
        let minNode = -1;

        for (let i = 1; i <= n; i++) {
            if (!visited.has(i) && dist[i] < minDist) {
                minDist = dist[i];
                minNode = i;
            }
        }

        if (minNode === -1) break;

        visited.add(minNode);

        for (const [neighbor, weight] of graph.get(minNode)) {
            dist[neighbor] = Math.min(dist[neighbor], dist[minNode] + weight);
        }
    }

    const maxDist = Math.max(...dist.slice(1));
    return maxDist === Infinity ? -1 : maxDist;
}

// Time: O((V + E) log V) with heap, O(V^2) simple
// Space: O(V + E)
```

---

24. Union Find (Disjoint Set Union)

Implementation

```
class UnionFind {
    constructor(n) {
        this.parent = Array.from({ length: n }, (_, i) => i);
        this.rank = new Array(n).fill(0);
        this.count = n;  // Number of connected components
    }

    find(x) {
        if (this.parent[x] !== x) {
            this.parent[x] = this.find(this.parent[x]);  // Path compression
        }
        return this.parent[x];
    }

    union(x, y) {
        const rootX = this.find(x);
        const rootY = this.find(y);

        if (rootX === rootY) return false;

        // Union by rank
        if (this.rank[rootX] < this.rank[rootY]) {
            this.parent[rootX] = rootY;
        } else if (this.rank[rootX] > this.rank[rootY]) {
            this.parent[rootY] = rootX;
        } else {
            this.parent[rootY] = rootX;
            this.rank[rootX]++;
        }

        this.count--;
        return true;
    }

    connected(x, y) {
        return this.find(x) === this.find(y);
    }

    getCount() {
        return this.count;
    }
}

// Time: O(alpha(n)) per operation, nearly O(1)
// Space: O(n)
```

---

25. Number of Connected Components

```
function countComponents(n, edges) {
    const uf = new UnionFind(n);
```

```
    for (const [u, v] of edges) {
        uf.union(u, v);
    }

    return uf.getCount();
}

// Alternative: DFS
function countComponentsDFS(n, edges) {
    const graph = new Map();
    for (let i = 0; i < n; i++) graph.set(i, []);
    for (const [u, v] of edges) {
        graph.get(u).push(v);
        graph.get(v).push(u);
    }

    const visited = new Set();
    let count = 0;

    function dfs(node) {
        visited.add(node);
        for (const neighbor of graph.get(node)) {
            if (!visited.has(neighbor)) {
                dfs(neighbor);
            }
        }
    }

    for (let i = 0; i < n; i++) {
        if (!visited.has(i)) {
            count++;
            dfs(i);
        }
    }

    return count;
}

// Time: O(V + E), Space: O(V + E)
```

---

26. Redundant Connection

Problem

Find the edge that can be removed to make a tree (cycle detection with Union Find).

```
function findRedundantConnection(edges) {
    const n = edges.length;
    const uf = new UnionFind(n + 1);  // 1-indexed

    for (const [u, v] of edges) {
        if (!uf.union(u, v)) {
            // Already connected, this edge creates cycle
            return [u, v];
        }
```

```
    }

    return [];
}

// Time: O(n * alpha(n)), Space: O(n)
```

Summary: Tree & Graph Patterns

Tree Patterns

Pattern	When to Use	Examples
DFS Recursion	Tree traversals, height, path	Max Depth, Path Sum
BFS Level Order	Level-by-level processing	Right Side View, Zigzag
Validate with Bounds	BST validation	Validate BST
LCA	Find common ancestor	LCA BST, LCA BT
Build from Traversals	Construct tree	Preorder + Inorder

Graph Patterns

Pattern	When to Use	Examples
DFS	Explore paths, connectivity	Islands, Clone Graph
BFS	Shortest path (unweighted)	Word Ladder, Rotting Oranges
Topological Sort	Dependencies, ordering	Course Schedule
Dijkstra	Shortest path (weighted)	Network Delay
Union Find	Connected components, cycles	Redundant Connection
Multi-source BFS	Multiple starting points	Rotting Oranges

Continue to Part 4: Dynamic Programming & Backtracking Solutions